

v2

Threat Hunting Professional

Introduction to Endpoint Hunting

Section 03 | Module 01

© Caendra Inc. 2020
All Rights Reserved

Table of Contents

MODULE 01 | INTRODUCTION TO ENDPOINT HUNTING

1.1 Introduction

1.2 Windows Processes

1.3 Endpoint Baselines

```
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

Introduction



1.1 Introduction

We will now focus our hunts on the endpoints.

At this point, imagine that threat intelligence feeds are in place and network traffic/flow is being monitored, but nothing is producing any alerts that would merit a hunt.

```
def initialize(experiment, observations = [], control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  freeze
  experiment.context
  experiment_name
  experiment_name
end

def match?
  # ...
end
```



1.1 Introduction

That doesn't mean the adversary has not obtained a foothold into the network. They probably found their way in by bypassing the perimeter defenses that are put into place.

Maybe an employee's laptop got compromised at their home or at a local coffee shop and the adversary found their way right into the juicy middle of our enterprise.

1.1 Introduction

To be successful as a hunter, you must already think that the adversary found its way in and you need to find them.

Remember we discussed **dwell time** in the beginning of this course. Based on reports, the adversary has been hidden in enterprise networks for months before detected.



1.1 Introduction

Servers typically don't change much, meaning software isn't constantly installed from day to day. The services, processes, etc. are not different from day to day. Windows patches might be done in a monthly or quarterly basis. Detecting anomalies on servers, if they're monitored, should be easier than on desktop machines.



1.1 Introduction

Each enterprise will have different policies in place that will either prohibit or allow the end user from performing certain tasks.

Some corporations will give end users local admin rights because it's easier to just allow the end user to install and configure anything rather than calling the help desk, or an application only works correctly if local admin rights are given to the user.

1.1 Introduction

Some corporations will give end users local admin rights only to certain folders, services, etc., but this can still pose a significant problem.

If this is known to an attacker, then that can be exploited to gain access and elevate privileges on the system.



1.1 Introduction

Enterprises, for the most part, are doing the best they can with what they have, with whatever appliances and budget they have in place.

At the end of the day, you still need to defend the network and find evil if they're within your systems.



1.1 Introduction

We'll begin by:

- Looking at the Windows operating system core processes
- Discussing the role each process plays
- Discussing the normal behavior of the process

This is important because even if you have appliances in place, attackers are becoming more and more crafty in tricking these appliances. We must recognize anything suspicious with these core processes, in case the appliance is unable to pick it up.

Windows Processes



1.2 Windows Processes

When it comes to the Windows core processes and attempting to detect if they are the legitimate core processes, we can look at a few key factors:

- Did the expected parent process spawn it?
- Is it running out of the expected path?
- Is it spelled correctly?
- Is it running under the proper SID?
- Is it signed by Microsoft?

1.2 Windows Processes

This is a snapshot of the processes running on a Windows system (VM). We will use this snapshot to explain the various core Windows processes.

- Name
- Purpose
- Executable path
- Parent process
- SID
- etc.

PID Command Line		

0	System	Idle Process
4	System	
264	smss.exe	
344	csrss.exe	
408	wininit.exe	
504	services.exe	
624	svchost.exe	
932	winPrvSE.exe	
688	vmacthlp.exe	
732	svchost.exe	
812	svchost.exe	
1160	audiodg.exe	
868	svchost.exe	
1812	"C:\Windows\system32\Dwm.exe"	
896	svchost.exe	
1620	taskeng.exe	
748	svchost.exe	
1096	svchost.exe	
1192	spoolsv.exe	
1224	svchost.exe	
1404	openvpnerv.exe	
1496	VGAuthService.exe	
1592	vmtoolsd.exe	
1632	ManagementAgentHost.exe	
1872	svchost.exe	
1924	svchost.exe	
928	dllhost.exe	
1852	msdtc.exe	
2804	svchost.exe	
3000	sppsvc.exe	
1036	svchost.exe	
2364	SearchIndexer.exe	
1716	svchost.exe	
2412	"taskhost.exe"	
1352	DiscSoftBusServiceLite.exe	
388	vmtoolsd.exe	
2612	PresentationFontCache.exe	
516	lsass.exe	
528	lsmd.exe	
400	csrss.exe	
1000	\\??C:\Windows\system32\conhost.exe	
664	\\??C:\Windows\system32\conhost.exe	
444	winlogon.exe	
2244	C:\Windows\Explorer.EXE	
2724	"C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" -n vmusr	

Windows 7

1.2.1 smss.exe

SMSS.EXE is known as the **Session Manager**. Its responsibility is to create new sessions.

- Session 0 starts csrss.exe and wininit.exe. (**OS services**)
- Session 1 starts csrss.exe and winlogon.exe. (**User session**)
- You will see **1 instance** (Session 0) within the process tree. The child instances of smss.exe which was used to create the other sessions, by copying itself into that new session, will self-terminate.
- Loads the registry and known DLLs into shared memory locations, among other things.

PID	Command Line
0	System Idle Process
4	System
264	smss.exe
344	csrss.exe
408	wininit.exe
504	services.exe
624	svchost.exe

400	csrss.exe
1000	\\?C:\Windows\system32\conhost.exe
664	\\?C:\Windows\system32\conhost.exe
444	winlogon.exe

1.2.1 smss.exe

Executable Path: %SystemRoot%\System32\smss.exe

Parent Process: System

Username: NT AUTHORITY\SYSTEM (S-1-5-18)

Base Priority: 11

Time of Execution: For Session 0, within seconds of boot time

1.2.1.1 Hunting Tip

Sessions 0 and 1 are normal. Additional sessions may be created by Remote Desktop Protocol (RDP) sessions and Fast User Switching on shared computers. If this does not apply to your environment, then it's worth checking the additional sessions (if such exist).

Remember only 1 instance of smss.exe should be running.

1.2.2 csrss.exe

CSRSS.EXE is the **Client/Server Run Subsystem Process**. It is responsible for managing processes and threads, as well as making the Windows API available for other processes. It's also responsible for mapping drive letters, creating temp files, and handling the shutdown process.

- Runs within Session 0 and 1.
- Will be available for each newly created user session.

PID	Command Line
0	System Idle Process
4	System
256	smss.exe
344	csrss.exe
408	wininit.exe
504	services.exe
624	svchost.exe

```
400 csrss.exe
1000 \\?\C:\Windows\system32\conhost.exe
664 \\?\C:\Windows\system32\conhost.exe
444 winlogon.exe
```

Windows 7

1.2.2 csrss.exe

Executable Path: %SystemRoot%\System32\csrss.exe

Parent Process: Created by child instance of SMSS.EXE but that process won't exist so will appear as no parent

Username: NT AUTHORITY\SYSTEM (**S-1-5-18**)

Base Priority: 13

Time of Execution: For Sessions 0 & 1, within seconds of boot time

1.2.2.1 Hunting Tip

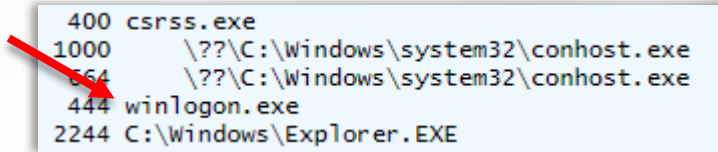
Malware authors can masquerade their malware to appear as this process by hiding in plain sight. They can name the malware as `csrss.exe` but just misspell it slightly. Examples of this would be `cssrs.exe`, `cssrss.exe`, and `csrsss.exe`.

Remember, typically you will see 2 instances of `csrss.exe`.

1.2.3 winlogon.exe

WINLOGON.EXE is the **Windows Logon Process**. It is responsible for user logons/logoffs. It launches LogonUI.exe for username and password and passes credentials to LSASS.exe which is verified via AD or local SAM.

- Loads Userinit.exe via **Software\Microsoft\Windows NT\CurrentVersion\Winlogon**.
 - Loads NTUSER.DAT into HKCU and starts the users shell via Userinit.exe.
 - Userinit initializes the user environment and runs logon scripts and GPO.



```
400 csrss.exe
1000 \??\C:\Windows\system32\conhost.exe
664 \??\C:\Windows\system32\conhost.exe
444 winlogon.exe
2244 C:\Windows\Explorer.EXE
```

Windows 7

*Both LogonUI.exe and Userinit.exe will exist and will not be visible after this process.

1.2.3 winlogon.exe

Executable Path: %SystemRoot%\System32\winlogon.exe

Parent Process: Created by child instance of SMSS.EXE but that process won't exist so will appear as no parent

Username: NT AUTHORITY\SYSTEM (**S-1-5-18**)

Base Priority: 13

Time of Execution: For Sessions 1, within seconds of boot time. Other instances may start later.

1.2.3.1 Hunting Tip

The abuse within this process often comes within the different components of the login process. Malware sometimes abuses the **SHELL** registry value. This value should be explorer.exe.

Another registry key that is abused by malware that works in conjunction with winlogon.exe is **Userinit**.

1.2.4 wininit.exe

WININIT.EXE is the **Windows Initialization Process**. It is responsible to launch **services.exe**, **lsass.exe**, and **lsmd.exe** in Session 0.

PID	Command Line
0	System Idle Process
4	System
264	smss.exe
344	csrss.exe
408	wininit.exe
504	services.exe
624	svchost.exe

Windows 7

1.2.4 wininit.exe

Executable Path: %SystemRoot%\System32\wininit.exe

Parent Process: Created by child instance of SMSS.EXE but that process won't exist so will appear as no parent

Username: NT AUTHORITY\SYSTEM (**S-1-5-18**)

Base Priority: 13

Time of Execution: Within seconds of boot time

1.2.4.1 Hunting Tip

You should only see **1 instance** of wininit.exe.

Remember the clues to look for provided at the beginning of this section, [here](#).

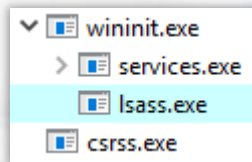
1.2.5 lsm.exe

LSM.EXE is the **Local Session Manager**. It is responsible to work with smss.exe to create, destroy, or manipulate new user sessions.

- Responsible for logon/logoff, shell start/end, lock/unlock desktop to name a few.

Note: After Windows 7, lsm.exe no longer exists, and it is now a service called lsm.dll.

Windows 10



Windows 7



408	wininit.exe
504	services.exe
624	svchost.exe
932	WmiPrvSE.exe
688	vmacthlp.exe
732	svchost.exe
812	svchost.exe
1160	audiodg.exe
868	svchost.exe
1812	"C:\Windows\system32\Dwm.exe"
896	svchost.exe
1620	taskeng.exe
748	svchost.exe
1096	svchost.exe
1192	spoolsv.exe
1224	svchost.exe
1404	openvpnserver.exe
1496	VGAuthService.exe
1592	vmtoolsd.exe
1632	ManagementAgentHost.exe
1872	svchost.exe
1924	svchost.exe
928	dllhost.exe
1852	msdtc.exe
2804	svchost.exe
3000	sppsvc.exe
1036	svchost.exe
2364	SearchIndexer.exe
1716	svchost.exe
2412	"taskhost.exe"
1352	DiscSoftBusServiceLite.exe
588	wmpnetwk.exe
2612	PresentationFontCache.exe
516	lsass.exe
528	lsm.exe

1.2.5 lsm.exe

Executable Path: %SystemRoot%\System32\lsm.exe

Parent Process: wininit.exe

Username: NT AUTHORITY\SYSTEM (S-1-5-18)

Base Priority: 8

Time of Execution: Within seconds of boot time

1.2.5.1 Hunting Tip

You should only see **1 instance** of `lsm.exe` on Windows 7 machines. You should **NOT** be seeing this on Windows 8 and beyond. It will be running as a service DLL instead, `lsm.dll`.

Remember the clues to look for provided at the beginning of this section, [here](#).

1.2.6 services.exe

SERVICES.EXE is the **Service Control Manager**. It is responsible for loading services (auto-start) and device drivers into memory.

- Parent to svchost.exe, dllhost.exe, taskhost.exe, spoolsv.exe, etc.
- Services are defined in **HKLM\SYSTEM\CurrentControlSet\Services**.
- Maintains an in-memory database of service information which can be queried using the built-in Windows tool, sc.exe.
- After a successful interactive login, services.exe will backup a copy of the registry keys into **HKLM\SYSTEM\Select\LastKnownGood** which will be known as the Last Known Good Configuration.



1.2.6 services.exe

Executable Path: %SystemRoot%\System32\services.exe

Parent Process: wininit.exe

Username: NT AUTHORITY\SYSTEM (S-1-5-18)

Base Priority: 9

Time of Execution: Within seconds of boot time

1.2.6.1 Hunting Tip

You should only see **1 instance** of services.exe. This is a protected process which makes it difficult to tamper with.

Remember the clues to look for provided at the beginning of this section, [here](#).

1.2.7 lsass.exe

LSASS.EXE is the **Local Security Authority Subsystem**. It is responsible for user authentication and generating access tokens specifying security policies and/or restrictions for the user and the processes spawned in the user session.

- Uses authentication packages within **HKLM\System\CurrentControlSet\Control\Lsa** to authenticate users.
- Creates security tokens for SAM, AD, and NetLogon.
- Writes to the Security event log.

Windows 7	
408	wininit.exe
504	services.exe
624	svchost.exe
932	WmiPrvSE.exe
688	vmacthlp.exe
732	svchost.exe
812	svchost.exe
1160	audiodg.exe
868	svchost.exe
1812	"C:\Windows\system32\Dwm.exe"
896	svchost.exe
1620	taskeng.exe
748	svchost.exe
1096	svchost.exe
1192	spoolsv.exe
1224	svchost.exe
1404	openvpnserver.exe
1496	VGAuthService.exe
1592	vmtoolsd.exe
1632	ManagementAgentHost.exe
1872	svchost.exe
1924	svchost.exe
928	dllhost.exe
1852	msdtc.exe
2804	svchost.exe
3000	sppsvc.exe
1036	svchost.exe
2364	SearchIndexer.exe
1716	svchost.exe
2412	"taskhost.exe"
1352	DiscSoftBusServiceLite.exe
988	wmpnetwk.exe
2612	PresentationFontCache.exe
516	lsass.exe
528	lsim.exe

1.2.7 lsass.exe

Executable Path: %SystemRoot%\System32\lsass.exe

Parent Process: wininit.exe

Username: NT AUTHORITY\SYSTEM (S-1-5-18)

Base Priority: 9

Time of Execution: Within seconds of boot time

1.2.7.1 Hunting Tip

You should only see **1 instance** of lsass.exe. This process is commonly attacked and abused by hackers and malware. It is targeted to dump password hashes and is often used to hide in plain sight. You might see different variations of spelling for this process (lass.exe or lsasss.exe), and might even see multiple instances of it, like with Stuxnet malware.

1.2.8 svchost.exe

SVCHOST.EXE is the **Generic Service Host Process**. It is responsible for hosting multiple services DLLs into a generic shared service process.

- Each service will have registry entries that include **ServiceDll**. This will instruct svchost.exe what DLL to use. The entry will also include **svchost.exe -k <name>**.
- Multiple instances of svchost.exe host will be running, as seen in the screenshot to the right.
 - All DLL-based services with the same <name> will share the same svchost.exe process.
 - <name> values are found in **Software\Microsoft\Windows NT\CurrentVersion\Svchost** registry key.
 - Each svchost.exe process will run with a unique -k <name>.

Windows 7	
408	wininit.exe
504	services.exe
624	svchost.exe
932	WmiPrvSE.exe
688	vmacthlp.exe
732	svchost.exe
812	svchost.exe
1160	audiodg.exe
868	svchost.exe
1812	"C:\Windows\system32\Dwm.exe"
896	svchost.exe
1620	taskeng.exe
748	svchost.exe
1096	svchost.exe
1192	spoolsv.exe
1224	svchost.exe
1404	openvpnerv.exe
1496	VGAuthService.exe
1592	vmtoolsd.exe
1632	ManagementAgentHost.exe
1872	svchost.exe
1924	svchost.exe
928	dllhost.exe
1852	msdtc.exe
2804	svchost.exe
3000	sppsvc.exe
1036	svchost.exe
2364	SearchIndexer.exe
1716	svchost.exe
2412	"taskhost.exe"
1352	DiscSoftBusServiceLite.exe
588	wmpnetwk.exe
2612	PresentationFontCache.exe
516	lsass.exe
528	lsme.exe

1.2.8 svchost.exe

Executable Path: %SystemRoot%\System32\svchost.exe

Parent Process: services.exe

Username: NT AUTHORITY\SYSTEM (**S-1-5-18**), LOCAL SERVICE (**S-1-5-19**), or NETWORK SERVICE (**S-1-5-20**) *

Base Priority: 8

Time of Execution: Varies

*In Windows 10, an instance will start as user upon login (-k UnistackSvcGroup).

1.2.8.1 Hunting Tip

This process is another process that is heavily abused. It can be used to launch malicious services (malware installed as a service). When this is done, (-k) will not be present. This process is often misspelled to hide in plain sight. Another technique used with this process is to place it in different directories, but note that services.exe will not be the parent.

Remember, the clues to look for provided at the beginning of this section, [here](#).

1.2.8.1 Hunting Tip

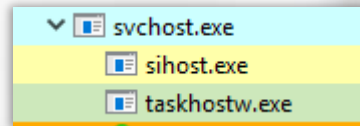
When it comes to services, we will need to perform extra steps to determine whether the service/DLL being loaded by svchost.exe is legitimate or not.

It's more than just checking for misspellings in svchost.exe, because techniques such as [Process Injection](#) and [Process Hollowing](#) can attack legitimate services. In these cases, advanced techniques are required, such as memory analysis.

1.2.9 taskhost.exe

TASKHOST.EXE is a **generic host process** which acts as a host for processes that run from DLLs rather than EXEs. At startup, TASKHOST checks the Services portion of the Registry to construct a list of DLL-based services that it needs to load, and then loads them.

- In Windows 8, this process was renamed to **taskhostex.exe**.
- In Windows 10, this process was renamed to **taskhostw.exe**.



Windows 10

Windows 7	
408	wininit.exe
504	services.exe
624	svchost.exe
932	WmiPrvSE.exe
688	vmacthlp.exe
732	svchost.exe
812	svchost.exe
1160	audiodg.exe
868	svchost.exe
1812	"C:\Windows\system32\Dwm.exe"
896	svchost.exe
1620	taskeng.exe
748	svchost.exe
1096	svchost.exe
1192	spoolsv.exe
1224	svchost.exe
1404	openvpnserver.exe
1496	VGAuthService.exe
1592	vmtoolsd.exe
1632	ManagementAgentHost.exe
1872	svchost.exe
1924	svchost.exe
928	dllhost.exe
1852	msdtc.exe
2804	svchost.exe
3000	sppsvc.exe
1036	svchost.exe
2364	SearchIndexer.exe
1716	svchost.exe
2412	"taskhost.exe"
1352	DiscSoftBusServiceLite.exe
588	wmpnetwk.exe
2612	PresentationFontCache.exe
516	lsass.exe
528	lsmon.exe

1.2.9 taskhost.exe

Executable Path: %SystemRoot%\System32\taskhost.exe

Parent Process: services.exe

Username: Varies

Base Priority: 8

Time of Execution: Varies

```
21 # @param: context
22 # @param: context - the experiment's context
23 # @param: observations - an array of Observations, in ascending
24 # @param: control - the control observation
25
26 def initialize(experiment, observations = [], control = null)
27   @experiment = experiment
28   @observations = observations
29   @control = control
30   @candidates = observations - [control]
31   evaluate_candidates
32
33   freeze
34 end
35
36 # @param: the experiment's context
37 def context
38   @experiment.context
39 end
40
41 # @param: the name of the experiment
42 def experiment_name
43   @experiment.name
44 end
45
46 # @param: the result a match between all
47 def match
48   @result
49 end
50
51 # @param: result
52 def result
53   @result
54 end
```

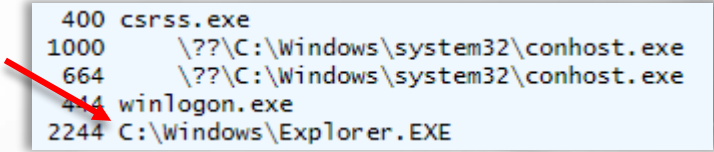
1.2.9.1 Hunting Tip

Remember, the clues to look for are provided at the beginning of this section, [here](#).

1.2.10 explorer.exe

EXPLORER.EXE is the **Windows Explorer**.

- Explorer.exe is responsible for the user's desktop and everything that comes with it, including access to files (file browser) and launching files via their file extensions.
- Even if multiple Windows Explorer windows open, only 1 process will be spawned per logged on user.



```
400 csrss.exe
1000   \??\C:\Windows\system32\conhost.exe
664   \??\C:\Windows\system32\conhost.exe
444   winlogon.exe
2244  C:\Windows\Explorer.EXE
```

Windows 7

1.2.10 explorer.exe

Executable Path: %SystemRoot%\explorer.exe

Parent Process: Created by USERINIT.EXE but that process won't exist so will appear as no parent

Username: As logged-on users

Base Priority: 8

Time of Execution: Varies

```
34 def initialize_experiment(experiment, observations = [], control = null)
35   @experiment = experiment
36   @observations = observations
37   @control = control
38   @candidates = observations + [control]
39   evaluate_candidates
40
41   freeze
42   end
43
44 # PUBLIC: the experiment's context
45 def context
46   @experiment.context
47 end
48
49 # PUBLIC: the name of the experiment
50 def experiment_name
51   @experiment.name
52 end
53
54 # PUBLIC: was the result a match between test and control?
55 def matched?
56   @test_result == @control_result
57 end
58
59 # PUBLIC: result of the test
60 def test_result
61   @test_result
62 end
63
64 # PUBLIC: result of the control
65 def control_result
66   @control_result
67 end
```

1.2.10.1 Hunting Tip

This process is targeted by malware as well. Different techniques will be incorporated, like the ones already mentioned, against this process. They will inject into the process, spawn malware named as explorer.exe, run it from a different folder or misspell it and have it run from the actual folder. Look for instances where explorer has CMD hanging off it or is listening/connected on a network port.



1.2.10.1 Hunting Tip

Let's add more to the [checklist](#) shown near the beginning of this section.

- Core Windows processes shouldn't run from Windows temp locations, or the Recycle Bin, and neither should be communicating to any outbound IPs.
- Check for digital signatures (all Microsoft artifacts should be digitally signed)

1.2.10.1 Hunting Tip

Let's add more to the [checklist](#) shown near the beginning of this section (CONTINUED):

- Look for any process that have cmd.exe, wscript.exe, powershell.exe etc. running as a child process.
- Lastly, you'll need to dig deeper, and that is where memory analysis will come into play to find instances of DLL injection, Process Hollowing, etc.

1.2 Windows Processes

Some additional references, besides the Internet, to learn more about these core Windows Processes:

- [Windows Internals, Seventh Edition, Part 1](#)
- [Windows Internals, Sixth Edition, Part 2](#)

***Windows Internals, Seventh Edition, Part 2 has not been released yet as of this date.**

<https://www.microsoftpressstore.com/store/windows-internals-part-1-system-architecture-processes-9780735684188>

<https://www.microsoftpressstore.com/store/windows-internals-part-2-9780735665873>

1.2 Windows Processes

This section was meant to get us familiar with the core Windows Processes to help us spot suspicious processes and/or:

- Services masquerading as legitimate processes
- Services actually using the legitimate processes
- Services being used to execute malware on the systems.

1.2 Windows Processes

Next, we need to be able to do the same after configuring these systems to be deployed in our environments.

The purpose of the next section is to look for suspicious artifacts hiding within processes, services, files, folders, etc.

Endpoint Baselines



1.3 Endpoint Baselines

In the next few slides, we'll discuss a technique that many enterprises should be implementing within their organization, **baselining**.

As mentioned in the Hunting Web Shells module, baselines will help you find anomalies within system processes, services, drivers, installed applications, file structures, etc.

```
24 # the experiment will result in 2
25 observations - an array of Observations, in this case
26 # control - the control observation
27
28 def initialize(experiment, observations = [], control = nil)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - [control]
33   evaluate_candidates
34
35   freeze
36
37   @context =
38     experiment.context
39
40     experiment_name
41     experiment_name
42
43     def fetcher
44       # ...
45     end
46
47     # ...
48   end
49
50   # ...
51   # ...
52   # ...
53   # ...
54   # ...
55   # ...
56   # ...
57   # ...
58   # ...
59   # ...
60   # ...
61   # ...
62   # ...
63   # ...
64   # ...
65   # ...
66   # ...
67   # ...
68   # ...
69   # ...
70   # ...
71   # ...
72   # ...
73   # ...
74   # ...
75   # ...
76   # ...
77   # ...
78   # ...
79   # ...
80   # ...
81   # ...
82   # ...
83   # ...
84   # ...
85   # ...
86   # ...
87   # ...
88   # ...
89   # ...
90   # ...
91   # ...
92   # ...
93   # ...
94   # ...
95   # ...
96   # ...
97   # ...
98   # ...
99   # ...
100  # ...
```

1.3 Endpoint Baselines

A baseline is a file that will be used for comparisons against current settings and/or configurations. You can compare the current state of a machine, file system, etc. against the baseline to determine anything out of place.

1.3 Endpoint Baselines

In the Hunting Web Shells module, we looked at how to use PowerShell to create a baseline of a folder structure on an internal web server.

In theory, the baseline would be created the last time a developer or dev team made updates to the folder structure. At the time of the comparison, whether daily, weekly, etc., that baseline (CSV) would be compared to the current folder structure (CSV) of the present day.

1.3 Endpoint Baselines

If a changed file or new file is detected, then that would be an indicator to investigate things on that web server more closely.

If your enterprise utilizes [change management](#), then there would be some type of proof that an update was scheduled to be conducted on that folder structure.

If there is no evidence of an authorized update, then you'll know something unauthorized took place, whether it's unintentional or intentional.

1.3 Endpoint Baselines

Alien Vault has a few good postings on their blog about the subject of File Integrity Monitoring that is worth reading. You can check:

- Part 1
- Part 2

<https://cybersecurity.att.com/blogs/security-essentials/what-is-file-integrity-monitoring>

<https://cybersecurity.att.com/blogs/security-essentials/what-is-file-integrity-monitoring-and-why-you-need-it>

1.3 Endpoint Baselines

There are several products that can assist with monitoring unauthorized changes to files. A few of these are included below:

- [TripWire](#)
- [SolarWinds](#)
- [AlienVault](#)

<https://www.tripwire.com/products/tripwire-file-integrity-manager/>
<https://www.solarwinds.com/security-event-manager/use-cases/file-integrity-monitoring-software>
<https://cybersecurity.att.com/solutions/pai-dss-file-integrity-monitoring>
<https://www.securetrust.com/solutions/compliance-technologies/file-integrity-monitoring/>
<https://logrhythm.com/solutions/security/file-integrity-monitoring/>

1.3 Endpoint Baselines

Detecting these changes and investigating each changed or new file in a large enterprise environment with hundreds of thousands of nodes is simply not feasible.

You will need some type of appliance to assist you with this task, like the ones mentioned in the previous slide.

1.3 Endpoint Baselines

Many appliances are able to assist us with the process of comparing artifacts to a set configuration or simply detecting changes that are not in a specified policy.

While vendors might refer to this technology with different terminology, security products on the market are able to do the job. At times, sadly, enterprises are not implementing these features within these appliances that already exist on their network **and they should be.**

1.3.1 System Center Configuration Manager

An example would be System Center Configuration Manager (SCCM) which is able to perform this [task](#):

“Configuration baselines in System Center Configuration Manager contain predefined configuration items and optionally, other configuration baselines. After a configuration baseline is created, you can deploy it to a collection so that devices in that collection download the configuration baseline and assess their compliance with it.”

1.3.1 System Center Configuration Manager

Why is SCCM listed as an example?

Many large enterprises might already have SCCM in their environment to deploy OS images, Windows Updates, push software packages, etc. As mentioned earlier, SCCM might not be utilized to its fullest potential. At times, the Security Team are not even aware of the full capabilities of the applications the other IT teams are using in the environment, SCCM being a prime example.



1.3.2 PowerShell Desired State Configuration

If you're on a tight security budget, and don't want to wait until next fiscal to put in for a new, much needed, security appliance, then you can resort to PowerShell. We already saw how PowerShell can help us with checking for changes in files.

PowerShell can also help us with configuration baselines on machines. This feature of PowerShell is known as **Desired State Configuration**.

```
18 def initialize_experiment, observations = []
19
20 @experiment = experiment
21 @observations = observations
22 @control = control
23 @candidates = observations - @control
24 evaluate_candidates
25
26 freeze
27
28 # TODO: the experiment's context
29 def context
30   experiment.context
31
32   experiment_name
33   experiment_name
34
35   def watcher
36     @watcher
37   end
38
39   @watcher.result
40 end
```

1.3.2 PowerShell Desired State Configuration

Desired State Configuration (DSC) is an essential part of the configuration, management, and maintenance of Windows-based servers.

It allows a PowerShell script to specify the configuration of the machine using a declarative model in a simple standard way that is easy to maintain and understand.

```
24 # The experiment will result in 2 observations
25 # observations - an array of observations, in this case
26 # control - the control observation
27
28 # Initialize the experiment, observations = {}, control = null
29
30 @experiment = experiment
31 @observations = observations
32 @control = control
33 @candidates = observations - @control
34 evaluate_candidates
35
36 freeze
37
38 # Context
39 experiment.context
40
41 # Experiment name
42 experiment_name
43
44 # Experiment name
45 experiment_name
46
47 # The result is a match between the result
48 # and the control
49
50 # The result is a match between the result
51 # and the control
```



1.3.2 PowerShell Desired State Configuration

Two of the main advantages of Desired State Configuration:

- To be able to configure machines identically with the aim of standardizing them.
- To ensure, at a given time, that the configuration of a machine will always be identical to its initial configuration, to avoid unauthorized changes.

```
23 # Experiment - the experiment name
24 # observations - an array of observations, in JSON
25 # control - the control observation
26
27 # Initialize experiment, observations = [], control = null
28
29 @experiment = experiment
30 @observations = observations
31 @control = control
32 @candidates = observations - $control
33 evaluate_candidates
34
35 freeze
36
37 experiment.context
38
39 # The experiment
40 experiment.name
41
42 # Experiment result
43 $experiment.result
```



1.3.2 PowerShell Desired State Configuration

If you're familiar with [Puppet](#), [Ansible](#), or [Chef](#) then you can grasp the concept behind DSC.

You can read more about DSC [here](#).

<https://puppet.com/>

<https://www.ansible.com/>

<https://www.chef.io/>

<https://www.red-gate.com/simple-talk/sysadmin/powershell/powershell-desired-state-configuration-the-basics/>

```
21 # PowerShell Script
22
23 # Experiment
24 # experiment - the experiment this script is for
25 # observations - an array of Observations, in ascending order
26 # control - the control Observation
27
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - @control
33   evaluate_candidates
34
35   freeze
36 end
37
38 # PUBLIC: the experiment's context
39 def context
40   @experiment.context
41 end
42
43 # PUBLIC: the name of the experiment
44 def experiment_name
45   @experiment.name
46 end
47
48 # PUBLIC: was the result a match between all candidates
49 def matched?
50   # ...
51   @is_matched?
52 end
```

1.3.3 Microsoft Security Compliance Manager

Another tool that can prove useful is [Microsoft Security Compliance Manager](#).

“SCM provides ready-to-deploy policies and DCM configuration packs based on Microsoft security guide recommendations and industry best practices, allowing you to easily manage configuration drift and address compliance requirements for Windows operating systems, Office applications, and other Microsoft applications.”

1.3.3 Microsoft Security Compliance Manager

You can download baselines for different Microsoft products.

They are all available [here](https://www.microsoft.com/en-us/download/details.aspx?id=55319).

```
def initialize(experiment, observations = [], control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  @candidates = observations - @control
  evaluate_candidates

  freeze
end

# Returns the experiment's context
def context
  @experiment.context
end

# Returns the name of the experiment
def experiment_name
  @experiment.name
end

# Returns whether the results match between all experiments
def matches?
  @experiment.result == 1
end
```

1.3.4 Microsoft Security Compliance Toolkit

For Windows 10, you can download the Microsoft Security Compliance Toolkit 1.0 [here](#).

As noted [here](#), Microsoft Security Compliance Manager is retired, but they mention the increasing popularity of PowerShell DSC, which was already mentioned.

<https://www.microsoft.com/en-us/download/details.aspx?id=55319>

<https://docs.microsoft.com/en-us/archive/blogs/secguide/security-compliance-manager-scm-retired-new-tools-and-procedures>

1.3 Endpoint Baselines

For certain types of analysis, such as memory analysis, it's good to have a process or service baseline to use to compare against a memory image to find evidence of malware running on a system.

In the next few slides, we'll look at how we can use PowerShell to create these baselines.

```
def initialize(experiment, observations = [], control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  @candidates = observations - {control}
  evaluate_candidates

  freeze
end

# TODO: the experiment's context
def context
  @experiment.context
end

# TODO: the experiment's name
def experiment_name
  @experiment.name
end

# TODO: the result of a match between an image and a baseline
def match?
  @baseline.match?(@image)
end
```

1.3.5 Services Baseline

To get a baseline of the running services on a newly imaged and configured machine, you can use the [Get-Service](#) PowerShell cmdlet.

```
Get-Service * | Where {$_.status -eq "Running"} | Export-Clixml Baseline-Services.xml
```

A brief explanation of the code (**only components not already explained**):

- `Get-Service *` instructs PowerShell to obtain all the services in the device.
- `Where {$_.status -eq "Running"}` is filtering those items and only pulling the running services and exporting the information to a XML file.

1.3.5 Services Baseline

To conduct the comparison we can simply use another PowerShell cmdlet, **Compare-Object**, to compare the original services baseline (XML) to the current list of running services on any given device.

1.3.5 Services Baseline

```
Compare-Object (Import-Clixml Baseline-Services.xml)(Get-Service * | Where {$_.status -eq "Running"}) -Property DisplayName | Where-Object {$_.sideindicator -eq "<="}
```

A brief explanation of the code (**only components not already explained**):

- Compare-Object instructs PowerShell to compare 2 sets of objects.
 - 1st object = Import-Clixml Baseline-Services.xml.
 - Here we're importing the baseline file (XML)
 - 2ND object = Obtaining the current list of running services
- -Property DisplayName is instructing PowerShell to pull the display name of the service
- Where-Object {\$_.sideindicator -eq "<="} is instructing PowerShell to display only the entries that Compare-Object is indicating as different from the baseline.

1.3.6 Processes Baseline

The same can be done with processes on a system. You just need to make some tweaks to the previous PowerShell code and instead of using Get-Service, we'll be using [Get-Process](#).

```
Get-Process | Export-Clixml Baseline-Processes.xml
```

A brief explanation of the code (**only components not already explained**):

- Get-Process instructs PowerShell to obtain all the processes in the device and export the information to a XML file.

1.3.6 Processes Baseline

To conduct the comparison, the process is similar with just a slight change as you can see in the below screenshot.

```
Compare-Object (Import-Clixml Baseline-Processes.xml)(Get-Process) -Property Name | Where-Object {$_.sideindicator -eq "<="}
```

If you need more information on this process, please refer to this blog [page](https://devblogs.microsoft.com/scripting/weekend-scripter-use-powershell-to-compare-two-snapshots-of-running-processes/) from **Hey, Scripting Guy!**.

1.3 Endpoint Baselines

You can create baselines on pretty much anything. The question is what do you want to baseline that you might need to run a comparison against in the future to detect something suspicious?

```
Get-WmiObject Win32_UserAccount | Export-Clixml Baseline-UserAccounts.xml
```

```
Get-WmiObject Win32_OperatingSystem | Export-Clixml Baseline-OS.xml
```

```
Get-WmiObject Win32_SystemUsers | Export-Clixml Baseline-SystemUsers.xml
```

1.3 Endpoint Baselines

Aside from the items already mentioned, here are a few items that should be baselined (if not controlled by Active Directory):

- Accounts on a system (user or service)
- Local administrators on a system
- Folder permissions
- Folders contents
 - Tasks folder (scheduled tasks)
 - Network folders containing internal installation executables & files

```

20 # Create the experiment context
21 att.registerExperiment(
22     "WarwickCreate", theExperimentContext)
23
24 # Experiment -> the experiment this function is from
25 @observations = an Array of Observations, in Experiment
26 @control    = the control Observation
27
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control      = control
32   @candidates  = observations - [control]
33   evaluate_candidates
34
35   freeze
36 end
37
38 # PSYSCI: the experiment's context
39 def context
40   experiment.context
41 end
42
43 # Public: the name of the experiment
44 def experiment_name
45   experiment.name
46 end

```

installation

to: /icomet/results/vb_1.1

1.3 Endpoint Baselines

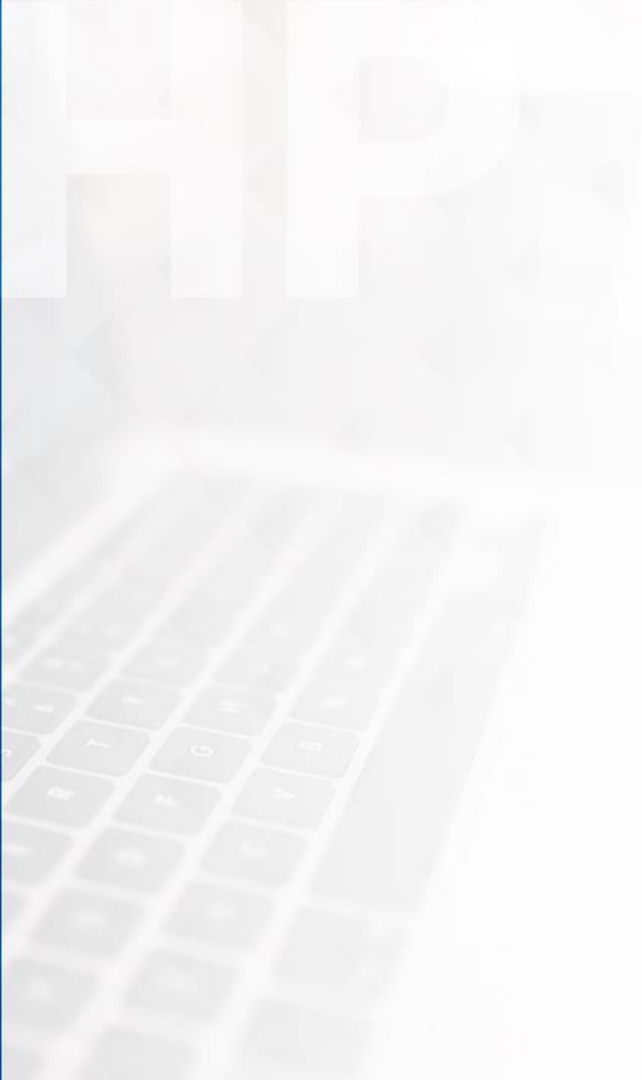
Refer to the MSDN [page](#) on **Win32 Classes** to obtain the different names of WMI classes you can access with the [Get-WMIObject](#) PowerShell cmdlet.

Creating baselines is an important security process that should be implemented whether you have enterprise-grade appliances, or you're using free tools such as PowerShell.

Module Conclusion

This concludes this module on Windows Processes and the importance of creating baselines. We have covered:

- Each of the core Windows Processes, what they do, how to detect that they are legitimate, and how malware attempts to abuse them.
- The importance of creating various types of baselines, and not neglecting tools that will aid you in the process of creating these much needed artifacts to help you on your hunts.



References



References

[Base Priority](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms685100\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms685100(v=vs.85).aspx)

[Process Injection](#)

<https://attack.mitre.org/wiki/Technique/T1055>

[Process Hollowing](#)

<https://attack.mitre.org/wiki/Technique/T1093>

[Windows Internals, Seventh Edition, Part 1](#)

<https://www.microsoftpressstore.com/store/windows-internals-part-1-system-architecture-processes-9780735684188>



References

[Windows Internals, Sixth Edition, Part 2](https://www.microsoftpressstore.com/store/windows-internals-part-2-9780735665873)

<https://www.microsoftpressstore.com/store/windows-internals-part-2-9780735665873>

[What is Change Management](https://www.prosci.com/change-management/what-is-change-management)

<https://www.prosci.com/change-management/what-is-change-management>

[What is File Integrity Monitoring?](https://cybersecurity.att.com/blogs/security-essentials/what-is-file-integrity-monitoring)

<https://cybersecurity.att.com/blogs/security-essentials/what-is-file-integrity-monitoring>

[What is File Integrity Monitoring and Why You Need It](https://cybersecurity.att.com/blogs/security-essentials/what-is-file-integrity-monitoring-and-why-you-need-it)

<https://cybersecurity.att.com/blogs/security-essentials/what-is-file-integrity-monitoring-and-why-you-need-it>



References

[TripWire](https://www.tripwire.com/products/tripwire-file-integrity-manager/)

<https://www.tripwire.com/products/tripwire-file-integrity-manager/>

[SolarWinds](https://www.solarwinds.com/security-event-manager/use-cases/file-integrity-monitoring-software)

<https://www.solarwinds.com/security-event-manager/use-cases/file-integrity-monitoring-software>

[File Integrity Monitoring Solutions](https://cybersecurity.att.com/solutions/pci-dss-file-integrity-monitoring)

<https://cybersecurity.att.com/solutions/pci-dss-file-integrity-monitoring>

[SecureTrust](https://www.securetrust.com/solutions/compliance-technologies/file-integrity-monitoring/)

<https://www.securetrust.com/solutions/compliance-technologies/file-integrity-monitoring/>



References

[LogRhythm](https://logrhythm.com/solutions/security/file-integrity-monitoring/)

<https://logrhythm.com/solutions/security/file-integrity-monitoring/>

[SCCM Baselines](https://docs.microsoft.com/en-us/sccm/compliance/deploy-use/create-configuration-baselines)

<https://docs.microsoft.com/en-us/sccm/compliance/deploy-use/create-configuration-baselines>

[Puppet](https://puppet.com/)

<https://puppet.com/>

[Anisble](https://www.ansible.com/)

<https://www.ansible.com/>



References

[Chef](https://www.chef.io/)

<https://www.chef.io/>

[PowerShell DSC](https://www.red-gate.com/simple-talk/sysadmin/powershell/powershell-desired-state-configuration-the-basics/)

<https://www.red-gate.com/simple-talk/sysadmin/powershell/powershell-desired-state-configuration-the-basics/>

[Microsoft SCM](https://www.microsoft.com/en-us/download/details.aspx?id=53353)

<https://www.microsoft.com/en-us/download/details.aspx?id=53353>

[Microsoft Windows Security Baselines](https://www.microsoft.com/en-us/download/details.aspx?id=55319)

<https://www.microsoft.com/en-us/download/details.aspx?id=55319>



References



[Microsoft Security Compliance Toolkit](https://www.microsoft.com/en-us/download/details.aspx?id=55319)

<https://www.microsoft.com/en-us/download/details.aspx?id=55319>

[Microsoft SCM](https://docs.microsoft.com/en-us/archive/blogs/secguide/security-compliance-manager-scm-retired-new-tools-and-procedures)

<https://docs.microsoft.com/en-us/archive/blogs/secguide/security-compliance-manager-scm-retired-new-tools-and-procedures>

[Get-Service](https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-service?view=powershell-7)

<https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-service?view=powershell-7>

[Compare-Object](https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/compare-object?view=powershell-7)

<https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/compare-object?view=powershell-7>



References

[Get-Process](https://msdn.microsoft.com/en-us/powershell/reference/5.1/microsoft.powershell.management/get-process)

<https://msdn.microsoft.com/en-us/powershell/reference/5.1/microsoft.powershell.management/get-process>

[Weekend Scripter: Use PowerShell to Compare Two Snapshots of Running Processes](https://devblogs.microsoft.com/scripting/weekend-scripter-use-powershell-to-compare-two-snapshots-of-running-processes/)

<https://devblogs.microsoft.com/scripting/weekend-scripter-use-powershell-to-compare-two-snapshots-of-running-processes/>

[Win32 Providers](https://docs.microsoft.com/en-us/windows/win32/wmisdk/wmi-classes)

<https://docs.microsoft.com/en-us/windows/win32/wmisdk/wmi-classes>

[Get-WmiObject](https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-wmiobject?view=powershell-5.1)

<https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-wmiobject?view=powershell-5.1>